FACULDADE SENAC PERNAMBUCO
Análise e Desenvolvimento de Sistemas

LEVANTAMENTO DETALHADO DE PROCESSO DE PROJETO

Projeto SIGO - Sistema Integrado de Gestão de Ocorrência (CBPM)

Do planejamento à conclusão: processos, metodologias, papéis, cronograma, indicadores, riscos e diagnóstico de maturidade

Autora: Bianca Guimarães, Gislany Araujo, João Marcos & Pedro Ayres
Recife, PE - 2026

Repositórios de referência:
github.com/BiancagscCabral/SIGO-CBPM (frontend)
github.com/Kacaii/senac-brigade-server (backend)

1. Apresentação do Projeto

O SIGO (Sistema Integrado de Gestão de Ocorrência) é uma aplicação web e mobile desenvolvida para modernizar o processo de registro de ocorrências do Corpo de Bombeiros (CBPM). O projeto foi realizado como Projeto Integrador do curso de Análise e Desenvolvimento de Sistemas da Faculdade Senac Pernambuco, com o objetivo de substituir os formulários físicos usados pelos bombeiros em campo por um fluxo digital, seguro e integrado a uma base de dados central.

A solução segue uma arquitetura cliente-servidor dividida em dois repositórios independentes: o frontend (SIGO-CBPM), responsável pela interface com a qual os bombeiros interagem, e o backend (senac-brigade-server), responsável por processar, validar e armazenar os dados.

2. Objetivos, Escopo e Características do Projeto

2.1 Objetivos principais

- Facilitar o registro e o gerenciamento digital das ocorrências atendidas pelas guarnições de bombeiros.
- Permitir acesso rápido a dados geográficos e informações do local da ocorrência, via geolocalização do navegador.
- Garantir integridade e segurança dos dados em todas as etapas do processo, da captura em campo até o armazenamento.
- Eliminar o retrabalho e a perda de informação associados ao preenchimento de formulários em papel.

2.2 Escopo do sistema

O escopo do SIGO cobre autenticação e recuperação de senha, um dashboard com visão geral e estatísticas, o registro de ocorrências com captura de geolocalização, listagem e visualização de ocorrências, um painel administrativo para gestão de usuários e equipes (brigadas), visualização de relatórios e configurações de perfil e de preferências de notificação. O sistema foi projetado para funcionar tanto em navegadores desktop quanto em dispositivos móveis, priorizando leveza e velocidade de acesso em campo.

2.3 Características técnicas

O SIGO é dividido em três camadas principais:

- Frontend: React + Vite, JavaScript (ES6+), consumindo uma API REST via Fetch API, com uso de recursos nativos do navegador como geolocalização (Navigator.geolocation) e armazenamento local (localStorage) para rascunhos e cache.
- Backend: escrito em Gleam, com o framework Wisp para expor a API REST, responsável por autenticação, regras de negócio e persistência.
- Banco de dados: PostgreSQL, acessado a partir do backend por meio da biblioteca Pog.

A infraestrutura conta com conteinerização via Docker e integração/deploy automatizados via GitHub Actions, com o ambiente de desenvolvimento padronizado em VSCode + WSL. O frontend está publicado

em produção (sigo-cbpm.vercel.app), e a comunicação entre camadas usa geocodificação reversa por meio da API pública Nominatim/OpenStreetMap para transformar coordenadas em endereços legíveis.

3. Etapas e Processos Utilizados no Desenvolvimento

Por se tratar de um Projeto Integrador multidisciplinar, o desenvolvimento seguiu um fluxo interativo típico de projetos acadêmicos orientados a produto, combinando levantamento de requisitos, prototipação, implementação incremental e integração contínua entre frontend e backend, que evoluíram em repositórios separados, mas de forma coordenada:

- Levantamento de requisitos e definição do problema: identificação da necessidade de digitalizar o registro de ocorrências do CBPM, com base no processo manual em papel utilizado anteriormente.
- Modelagem e design: definição da identidade visual e da experiência de uso (UI/UX) conduzida pela Direção de Projeto/P.O., seguida de prototipação de telas antes da implementação.
- Definição de arquitetura: separação clara entre frontend (React/Vite) e backend (Gleam/Wisp), com contrato de API REST definido entre as equipes para permitir desenvolvimento paralelo.
- Modelagem de dados: elaboração do Diagrama de Entidade-Relacionamento cobrindo usuários, brigadas (equipes), participação em brigadas, ocorrências e o vínculo ocorrência–brigada, conduzida pela equipe de backend.
- Implementação incremental: construção dos módulos por funcionalidade (autenticação, dashboard, registro de ocorrências, administração de usuários/equipes, relatórios, configurações), refletida no alto número de commits registrados (134 no frontend e 432 no backend).
- Integração contínua: consumo dos endpoints REST do backend pelo frontend através de um proxy configurado no Vite, permitindo que as duas equipes trabalhassem de forma desacoplada.
- Testes e ajustes: verificação funcional das telas e dos fluxos de ponta a ponta (login, criação de ocorrência, listagem, painel administrativo), com correções registradas como issues abertas no repositório.
- Deploy e apresentação: publicação da aplicação frontend (Vercel) e do backend containerizado, com pipeline de CI/CD via GitHub Actions, culminando na apresentação do Projeto Integrador.

4. Metodologias, Técnicas e Ferramentas Empregadas

4.1 Metodologia de trabalho

O projeto foi conduzido com apoio de uma Scrum Master formalmente designada na equipe, o que indica adoção de práticas ágeis (inspiradas em Scrum/Kanban) para organização do trabalho em ciclos curtos, com backlog de funcionalidades e acompanhamento incremental do progresso padrão recorrente nos Projetos Integradores da Faculdade Senac, que Bianca também aplicou em outros semestres com uso de quadros Kanban/Trello.

4.2 Técnicas de desenvolvimento

- Componentização de UI: interface dividida em componentes reutilizáveis (src/components/) e páginas (src/pages/).

- Gerenciamento de estado global via React Context API (src/contexts/), compartilhando dados de usuário e ocorrências entre telas.
- Camada de serviço isolada (src/services/) para centralizar as chamadas à API, separando lógica de dados da lógica de apresentação.
- Roteamento centralizado (routes.jsx) com proteção de rotas autenticadas via componente ProtectedRoute.jsx.
- Hooks customizados para lógica reutilizável, incluindo um hook dedicado a acessibilidade por controle de voz (useSpeech.js).
- Modelagem de dados relacional com Entity-Relationship Diagram formalizado antes da criação das tabelas no PostgreSQL.

4.3 Ferramentas e tecnologias

Categoria	Ferramentas / Tecnologias
Frontend	React, Vite, JavaScript (ES6+), CSS por componente
Backend	Gleam, Wisp (framework REST)
Banco de dados	PostgreSQL, biblioteca Pog
Infraestrutura	Docker, GitHub Actions (CI/CD)
Ambiente de desenvolvimento	VSCode, WSL (Windows Subsystem for Linux)
APIs de navegador	Fetch API, localStorage, Navigator.geolocation
APIs externas	Nominatim / OpenStreetMap (geocodificação reversa)
Controle de versão	Git / GitHub (repositórios separados para frontend e backend)
Publicação	Vercel (frontend)

5. Papéis e Responsabilidades dos Envolvidos

A equipe do SIGO foi composta por seis integrantes, organizados em três frentes: gestão de projeto, desenvolvimento frontend e desenvolvimento backend.

Integrante	Papel
Thaise Renaux	Direção de Projeto / Product Owner / Design (UI/UX)
Bianca Guimarães	Desenvolvedora Frontend
Gislany Araujo	Desenvolvedora Frontend / Design (UI/UX) / Scrum Master
João Marcos	Desenvolvedor Frontend
Pedro Ayres	Desenvolvedor Backend
Lucas Eloi	Desenvolvedor Backend

5.1 Observação do Papel de cada Integrante

Bianca Guimarães (Desenvolvedora Frontend) : além da implementação de telas React, sua atuação como desenvolvedora frontend envolveu consumo dos endpoints REST do backend, participação nas decisões de estrutura de componentes/contextos e colaboração direta com a equipe de backend para alinhar contratos de API - competência que ela também destaca em seu portfólio pessoal como projeto de referência em React/Vite.

Thaise Renaux (Direção de Projeto / P.O. / Design UI-UX): concentrou a responsabilidade de definir a visão de produto e priorizar funcionalidades, além de conduzir o design de interface que orientou a implementação do frontend. Por acumular direção de projeto e product ownership, foi o ponto central de decisão sobre escopo — o que também explica por que ajustes de payload e novas telas dependiam de alinhamento direto com ela antes de entrarem em desenvolvimento.

Gislany Araujo (Desenvolvedora Frontend / Design UI-UX / Scrum Master): acumulou três frentes implementação de telas, apoio ao design e facilitação do processo ágil da equipe. Esse acúmulo de papéis é comum em equipes acadêmicas pequenas, mas concentra risco de sobrecarga: dificuldades de priorização ou de condução das cerimônias ágeis tendem a impactar diretamente a própria capacidade de entrega de código dela.

João Marcos (Desenvolvedor Frontend): atuou como o terceiro desenvolvedor da camada de interface, contribuindo para a divisão do trabalho entre autenticação, dashboard, registro de ocorrências, painel administrativo e demais telas. A ausência de registros públicos individualizados por autor de commit não permite detalhar com precisão quais módulos couberam a cada um dos três frontends, mas a distribuição em componentes, páginas, contextos e hooks sugere um time capaz de trabalhar em paralelo sem muita colisão de código.

Pedro Ayres (Desenvolvedor Backend): junto com Lucas Eloi, foi responsável pela API REST em Gleam/Wisp, pela modelagem do banco em PostgreSQL e pelas regras de autenticação e negócio. Além de responder às issues de ajuste de payload abertas pelo próprio time de backend, indício de que Pedro e Lucas também atuaram na revisão e evolução da API a partir do feedback do frontend.

Lucas Eloi (Desenvolvedor Backend): compartilhou com Pedro Ayres a responsabilidade pela camada de dados e pela API, incluindo a definição do Diagrama de Entidade-Relacionamento (usuários, brigadas, participação em brigadas, ocorrências e o vínculo ocorrência-brigada). A dupla de backend também foi autora

6. Planejamento, Cronograma e Recursos Utilizados

O planejamento seguiu o calendário do semestre letivo em que o Projeto Integrador foi desenvolvido, com entregas parciais (protótipo, MVP funcional, versão final) alinhadas às datas de acompanhamento e Demoday da Faculdade Senac modelo também utilizado por Bianca em projetos de outros semestres (como o projeto Oracle EPM Planning, apresentado em Demoday).

6.1 Marcos de desenvolvimento observáveis

- Definição de escopo e arquitetura geral do sistema (cliente-servidor, contrato de API).
- Modelagem do banco de dados e construção da API REST no backend (maior volume de commits, refletindo a estruturação de rotas, autenticação e regras de negócio).

- Construção da base do frontend: roteamento, autenticação e layout padrão (cabeçalho e barra lateral).
- Implementação incremental das funcionalidades principais: dashboard, registro de ocorrências com geolocalização, listagem/visualização, painel administrativo e relatórios.
- Integração final entre frontend e backend, testes de fluxo completo e ajustes de deploy.
- Publicação da aplicação e apresentação final do projeto.

6.2 Recursos utilizados

- Recursos humanos: seis integrantes distribuídos entre gestão/design, frontend e backend.
- Recursos tecnológicos: ambientes de desenvolvimento individuais (VSCode + WSL), repositórios Git hospedados no GitHub, pipeline de CI/CD via GitHub Actions e hospedagem gratuita para o frontend (Vercel).
- Recursos de conhecimento: documentação da API mantida no README do backend e replicada como referência de endpoints no README do frontend, reduzindo a dependência de comunicação constante entre as duas frentes.

Por não haver, nos repositórios públicos, um cronograma formal (Gantt, planilha de sprints ou board do Trello/GitHub Projects com datas), este item foi reconstruído a partir da sequência de commits e da estrutura de entregas típica de Projetos Integradores. Recomenda-se que Bianca complemente esta seção com datas reais de início, sprints e entrega final, caso tenha esse registro em atas, planilhas ou no board do time.

7. Indicadores, Métricas e Resultados Obtidos

Indicador	Frontend (SIGO-CBPM)	Backend (senac-brigade-server)
Commits registrados	134	432
Issues abertas no encerramento da consulta	6	5
Estrelas no GitHub	4	3
Forks	1	2
Licença	MIT	MIT
Deploy	Produção (Vercel)	Containerizado (Docker) / CI-CD via GitHub Actions

Em termos funcionais, o resultado obtido foi uma aplicação operacional com 21 endpoints REST mapeados e consumidos, cobrindo autenticação, administração de usuários e equipes, dashboard de estatísticas, gestão de ocorrências e preferências de notificação. O volume consideravelmente maior de commits no backend (432 contra 134 no frontend) é coerente com o esforço de modelagem de dados, regras de negócio e construção de todas as rotas da API, enquanto o frontend concentrou o esforço em consumo dessas rotas e experiência do usuário.

Como resultado qualitativo, o projeto evoluiu de um MVP com funcionalidades essenciais (login, registro de ocorrência) para uma aplicação com módulos administrativos completos, recursos de acessibilidade

(controle por voz, ajuste de tema e tamanho de texto) e persistência local de rascunhos indicando amadurecimento do escopo ao longo do desenvolvimento.

8. Registros e Documentos Produzidos

- README técnico do frontend, descrevendo objetivos, arquitetura geral e do frontend, funcionalidades implementadas, tecnologias, endpoints consumidos e instruções de execução (documento robusto, com sumário navegável).
- README técnico do backend, contendo diagrama de arquitetura, tabela completa de rotas da API (método, descrição, formato de payload) e o Diagrama de Entidade-Relacionamento (ERD) do banco de dados em notação Mermaid.
- Diagrama de arquitetura do backend (imagem versionada no repositório).
- Modelo de dados formal (ERD), cobrindo as entidades user_account, notification_preference, brigade, brigade_membership, occurrence e occurrence_brigade.
- Issues no GitHub, funcionando como registro de bugs, solicitações de mudança e pendências técnicas (ex.: bug de redirecionamento no deploy, solicitação de dashboard de análise de dados, ajuste de payload de endpoint).
- Configurações de containerização e pipeline (dockerfile, justfile, workflows do GitHub Actions), documentando o processo de build e deploy.
- Arquivo de licença (MIT) em ambos os repositórios, formalizando os termos de uso e distribuição do código.

9. Problemas, Riscos e Dificuldades Identificados

Os problemas e pendências abaixo foram levantados a partir das issues abertas nos dois repositórios no momento da consulta, refletindo o estado real do backlog do projeto:

Repositório	Issue	Natureza
Frontend	Dashboard para Análise de Dados	Funcionalidade pendente / melhoria
Frontend	Mudanças no endpoint de criar novas equipes	Ajuste de integração com backend
Frontend	Acessar lista completa de ocorrências	Funcionalidade pendente
Frontend	Substituir UUID por Seleção Visual	Melhoria de usabilidade (enhancement)
Frontend	Redirecionamento incorreto no deploy	Bug em produção
Frontend	Atualizar payload GET /user/{id}/crew_members	Ajuste de contrato de API

Esse conjunto de pendências evidencia riscos típicos de projetos com duas equipes trabalhando em repositórios separados: divergências pontuais entre o payload esperado pelo frontend e o efetivamente retornado pelo backend (risco de acoplamento fraco mal sincronizado), necessidade de exposição de mais dados via API à medida que novas telas são construídas, e um bug de redirecionamento identificado apenas

em ambiente de produção sinal de que a cobertura de testes end-to-end antes do deploy pode ser reforçada.

Do ponto de vista de dificuldades de processo, a ausência de um cronograma versionado publicamente (datas de sprint, marcos formais) é, em si, uma fragilidade de rastreabilidade: dificulta reconstruir o histórico de decisões e estimar desvios de prazo a posteriori, algo relevante para este próprio diagnóstico de maturidade.

10. Diagnóstico de Maturidade do Projeto

Com base nas evidências reunidas documentação técnica, estrutura de código, práticas de controle de versão, presença de papéis definidos e uso de issues para gestão de pendências, o SIGO pode ser posicionado em um nível de maturidade inicial-a-gerenciado, típico de projetos acadêmicos bem conduzidos, mas ainda distante da maturidade de processos de uma organização de TI consolidada.

10.1 Pontos fortes identificados

- Papéis claramente definidos (P.O., Scrum Master, desenvolvedores frontend e backend), indicando organização mínima de equipe além do simples "todos fazem tudo".
- Separação arquitetural limpa entre frontend e backend, com contrato de API documentado prática que aproxima o projeto de padrões profissionais de integração entre times.
- Documentação técnica acima da média para projetos acadêmicos: README com arquitetura, endpoints, instruções de execução e ERD versionado.
- Uso ativo do sistema de issues do GitHub para registrar bugs e melhorias, em vez de depender só de comunicação informal.
- Pipeline de CI/CD e containerização configurados, aproximando o projeto de práticas de DevOps mesmo em escala acadêmica.

10.2 Lacunas e oportunidades de melhoria

- Ausência de cronograma formal versionado (datas de sprint, marcos, planejamento de capacidade), o que limita o rastreamento de prazos e a análise de desvios.
- Falta de métricas de qualidade automatizadas (cobertura de testes, testes automatizados de integração entre frontend e backend), o que provavelmente explica bugs identificados apenas em produção.
- Dependência de sincronização manual de contrato de API entre equipes, evidenciada pelas issues de ajuste de payload - poderia ser mitigada com um contrato de API versionado (ex.: OpenAPI/Swagger) compartilhado desde o início.
- Pendência de funcionalidades de análise de dados (dashboard analítico), sinalizando que o escopo de Business Intelligence do produto ainda não foi totalmente entregue.

10.3 Classificação de maturidade

Numa leitura inspirada nos níveis do CMMI (Capability Maturity Model Integration), adaptados à escala de um Projeto Integrador acadêmico, o SIGO se posiciona predominantemente no Nível 2 - Gerenciado: existem papéis, processos de desenvolvimento repetíveis (componentização, camada de serviço, ERD, CI/CD) e rastreamento básico de problemas via issues, mas ainda faltam processos padronizados e medidos

de forma sistemática (Nível 3), como métricas de qualidade formalizadas, cronograma versionado e contrato de API formalmente especificado. O projeto demonstra, portanto, maturidade de processo compatível com uma equipe que já superou o estágio puramente ad hoc (Nível 1), mas que ainda tem espaço claro de evolução rumo a práticas mais previsíveis e mensuráveis.

Nota metodológica: este diagnóstico foi construído a partir de evidências públicas disponíveis nos dois repositórios (READMEs, arquitetura, contagem de commits, issues abertas, autores e licenças) no momento da consulta. Itens como cronograma detalhado, atas de reunião, critérios de aceite formais e métricas internas de equipe não estão versionados publicamente; caso Bianca tenha acesso a esses registros (quadro Kanban/Trello do projeto, planilhas de sprint, atas), recomenda-se incorporá-los para enriquecer as seções 6, 7 e 8 com dados primários.